Web Science

Assignment – 2

Institute of Web Science and Technologies

Universität Koblenz-Landau

Group name: FOWLER

Name: Jeevan Saraf

jeevansaraf@uni-koblenz.de

Matriculation no: **222100445**

Name: Parth Chavda

parthchavda@uni-koblenz.de

Matriculation no: **222100495**

Name: Prajna Shetty

prajnashetty73@uni-koblenz.de

Matriculation no: **222100479**

Name: Ritika Bhandari

rbhandari@uni-koblenz.de

Matriculation no: **222100563**

1 Challenges

Since file sizes are mostly not of the order of a few kilobytes, what would the challenges be if it were to be split and then sent across information channel?
Your answers should show the priority of challenges and explain it in your own words
why they have been so ordered.
(Do not copy what has been told in the videos.)

Answer: When the size of the file is large and needs to be sent across the information channel, the file is split into different packets and are sent over the network. During this process, there are multiple challenges that need to be addressed so that the correct file is received at the receiver's end. The challenges are listed below:

- The first challenge is in-order delivery of the file. When the file is split and sent to the receiver end, the split packets should be arranged in order before the receiver receives it otherwise the receiver will receive the incorrect file. For instance, if LIVE is split into 4 different packets L, I, V, and E and if there are not in order when the receiver receives it (EVIL), it can change the whole meaning as well as the whole context.
- Another challenge is the end-to-end delivery of the file. When multiple applications are running in one network and there are multiple files that are sent to another host in another network, then the delivery of the packets to the correct application is important. So, from where the packets are sent from one machine to where and which application on another machine it should receive is challenging.
- Another issue is controlling the flow of data. If the sender and receiver have different capacities to send and receive the data and the sender sends packets more than the receiver could receive, then the receiver won't be able to receive the whole file and there would be a loss of data.
- Another issue is to control the congestion by taking care of the information channel through which file splits are sent.
- When file splits are sent, there should not be any loss of file split/data else the reliability of data is will not be secured.
- The file splits should be free from error so that the accuracy of file is maintained at the receiver ends.

2 URL Parsing

Write a python script that has a function called urlparser.

def urlparser (url):

This function will accept a URL from the list of two URLs and parse it into its constituent parts.
You may use a dictionary to store the parsed portions of the url.
You are not allowed to use any specific libraries that help in url parsing.
For testing the robustness of your code, you may use any valid arbitrary URL.

CODE:

```
import re

mylist =
['https://localhost:8080/search?q=how+to+dispose+of+a+corpse&oq=how+to+dispose+of+a+c
orpse&aqs=chrome..69i57j69i64.4925j1j7&sourceid=chrome&ie=UTF-8',
  'http://www.google.com/101-things-to-do-with-a-dead-
body_9780997711639?utm_source=google-shopping&utm_medium=cpc#dfsdfj8877']
def url_parser(s):

  try:
    scheme = re.findall('(\w+)://', s)[0]
  except IndexError: return "Enter the complete URL"

  try:
    host = re.findall('://([\da-z\.-]+)/?', s)
  except IndexError: host = ''

  port = []
  try:
    port = re.findall('://[\da-z.]*:([0-9]+)/?',s)[0]
  except IndexError:
    if not port:
      if scheme == 'http': port = '80'
```

```
        elif scheme == 'https' : port = '443'
        else: port = ''


    try:
        path = re.findall('\w(/[\d\w\-\_/]+)', s)[0]
    except IndexError: path = ''

    try:
        query = re.findall('\?([\w=\+&._:-]+)', s)[0]
    except IndexError: query = ''

    try:
        fragment = re.findall('#([\w=&._-]+)', s)[0]
    except IndexError: fragment = ''


    elements = {
        'scheme': scheme ,
        'host':  host,
        'port' : port ,
        'path': path ,
        'query': query,
        'fragment': fragment
    }

    return elements

for x in mylist:
    print(url_parser(x))
```

3 Communication

3.1 Hand-shake

Explain in steps, the process of a TCP three-way handshake.

Answer: TCP three-way handshake could also be seen as a way of how TCP connection is established. TCP provides reliable communication with PAR(Positive Acknowledgement with Re-transmission). A device using PAR resend the data unit until it receives an acknowledgement. If receiver receives damages data, TCP-IP will recognise it is damaged by Error Detection and receiver discards the segment. So, sender has to resend the data if positive acknowledgement is not received. Here, communication is done three times between sender and receiver for a reliable TCP connection to get established. This is how it works(in steps) :

- **Step 1 (SYN):** The client wants to establish a connection with a server, so it sends a segment with SYN(Synchronize Sequence Number) to the IP address of the server, which informs the server that the client is likely to start communication.

- **Step 2 (SYN + ACK):** Server responds to the client request with SYN-ACK signal bits set. Acknowledgement(ACK) signifies the response of the segment it received. ACK message is set to one higher than the sequence number. The server also sends the maximum segment size and size of the window that will be used. SYN signifies with what sequence number it is likely to start the segments with. Sequence number of SYN message will be different than that of client.

- **Step 3 (ACK):** In the final part, client acknowledges the response of the server and sends ACK message which includes the SYN number of server (augmented by one), that they both establish a reliable connection with which they will start the actual data transfer.

3.2 Sliding Window

Explain briefly the concept of Sliding Window Protocol. Discuss which type of Sliding Protocol has higher efficiency.

Answer: The sliding window is a technique for sending multiple frames at a time. The sliding window is used in Transmission Control Protocol. In this technique, each frame is sent from the sequence number. The sequence numbers are used to find the missing data in the receiver end. The purpose of the sliding window technique is to avoid duplicate data, so it uses the sequence number. Sliding window protocol has two types:

1. Go-Back-N ARQ

2. Selective Repeat ARQ

Go-Back-N ARQ protocol is also known as Go-Back-N Automatic Repeat Request. It is a data link layer protocol that uses a sliding window method. In this, if any frame is corrupted or lost, all subsequent frames have to be sent again. The size of the sender window is N in this protocol. For example, Go-Back-8, the size of the sender window, will be 8. The receiver window size is always 1.If the receiver receives a corrupted frame, it cancels it. The receiver does not accept a corrupted frame. When the timer expires, the sender sends the correct frame again.

Selective Repeat ARQ is also known as the Selective Repeat Automatic Repeat Request. It is a data link layer protocol that uses a sliding window method. In this protocol, the size of the sender window is always equal to the size of the receiver window. The size of the sliding window is always greater than 1. If the receiver receives a corrupt frame, it does not directly discard it. It sends a negative acknowledgment to the sender. The sender sends that frame again as soon as on the receiving negative acknowledgment. There is no waiting for any time-out to send that frame.

The Go-back-N ARQ protocol works well if it has fewer errors. But if there is a lot of error in the frame, lots of bandwidth loss in sending the frames again. So, we use the Selective Repeat ARQ protocol. In particular, if no packets are damaged, selective repeat and Go Back N perform equally well.

3.3 Protocol and Traffic

Explain connection oriented and connection less protocol. Also explain solicited and unsolicited traffic.

Answer: Both connection-oriented protocol and connection less protocol are used for the connection establishment between two or more devices.

Connection-oriented protocol is like telephone system. It includes connection establishment and connection termination. In a connection-oriented protocol, the Handshake method is used to establish the connection between sender and receiver.

Connection-less protocol is like the postal system. It does not include any connection establishment and connection termination. Connection-less protocol does not give a guarantee of reliability. In this, Packets do not follow the same path to reach their destination.

Solicited traffic is traffic that was initiated by you. Solicited traffic automatically gets a pass, no matter the port, because you initiated it. While in unsolicited traffic, receiver receives data or packet from sender without receiver sending request for that information. But, it is up to you, whether you want to receive it or block it.

4 TCP Client - Server (10 Points)

In Question 2, you found out how to parse a given URL and store the different parts in a dictionary.

For this question you have the following tasks:

1. Write a simple TCP Client that sends the different parts of the given URL in a JSON1 package.

CODE:

```
import socket
import json

message = {
   'scheme': 'https',
   'host': 'www.facebook.com',
   'port': 443,
   'path': 'photo.php',
   'query': 'fbid=2068026323275211&set=a.269104153167446&type=3&theater',
   'fragment': '1efh3',
}


def client_program():
   host = socket.gethostname()
   port = 5001


   data = json.dumps(message)
   client_socket = socket.socket()

   try:
```

```
        client_socket.connect((host, port))
        client_socket.send(data.encode())

        received = client_socket.recv(1024)
        received = received.decode("utf-8")

    finally:
        client_socket.close()


    print("Connection Closed.!")

if __name__ == '__main__':
    client_program()
```

2. Write a simple TCP Server that receives the message from the TCP Client and assemble the parts of the URL to print the entire valid URL.

CODE:

```
import socket
import json

def server_program():
    host = socket.gethostname()
    port = 5001

    server_socket = socket.socket()

    server_socket.bind((host, port))

    server_socket.listen(2)
    conn, address = server_socket.accept()
    print("Connection from: " + str(address))
    data = conn.recv(1024)
    data = data.decode('utf-8')
    data = json.loads(data)
```

```
    url = str(data['scheme']) + '://' + str(data['host']) + ':' + str(data['port']) + '/' + str(data['path']) +
'?' + str(data['query']) + '#' + str(data['fragment'])
    print("from Client: ",url)
    conn.close()

if __name__ == '__main__':
    server_program()
```